Solving the max-cut problem with QAOA

Bruno Fédrici

September 15, 2025

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid iterative method for solving combinatorial optimization problems. In this tutorial, we demonstrate how to implement the QAOA algorithm using *Qiskit 2.x* for solving a simple max-cut problem.

1 Graphs and cuts

For a graph $G(V, E)$, a maximum cut is a cut whose size is at least the size of any other cut. That is, it is a partition of the graph's vertices into two complementary sets S and T, such that the number of edges between S and T is as large as possible. Finding such a cut is known as the max-cut problem and is NP-hard. The max-cut problem can be expressed as the following minimization problem:

$$\begin{aligned} &\text{Minimize} && \sum_{(i,j)\in E} z_i z_j \\ &\text{subject to} && z_j \in \{-1, 1\}, \qquad j = 0, \ldots, n-1 \end{aligned}$$

where n is the number of vertices, and $z_j = -1$ (resp 1) if vertex j belongs to S (resp T) (or vice versa).

2 QAOA

The QAOA algorithm is a layered algorithm involving both a cost Hamiltonian H_C, that is analogous to the cost function of our optimization problem, and a mixer Hamiltonian H_M which is non-commutative with H_C. By first initializing the qubits in the ground state of H_M, the QAOA algorithm then makes the system evolving to the ground sate of H_C by applying the following sequence of unitary evolution:

$$U_{ansatz} = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \ldots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C}$$

where the optimal betas and gammas are to be found through using a classical optimizer.

A common choice for the mixer Hamiltonian is

$$H_M = -\sum_{i\in V} X_i$$

The ground state of this Hamiltonian is $|+\rangle^{\otimes n}$, and so initializing the qubits in this state is no more simple than applying a Hadamard gate on every qubit.

Having done that, applying the QAOA ansatz makes the qubit register evolving to final state

$$|\psi_p(\overrightarrow{\gamma}, \overrightarrow{\beta})\rangle = U_{ansatz} |+\rangle^{\otimes n}$$

from which we can finally evaluate

$$\langle H_C \rangle = \langle \psi_p(\overrightarrow{\gamma}, \overrightarrow{\beta})| H_C |\psi_p(\overrightarrow{\gamma}, \overrightarrow{\beta})\rangle$$

To minimize $\langle H_C \rangle$, starting from a random distribution of the gammas and the betas, a classical optimizer is used so as to find the optimal parameter settings. A summary of the algorithm workflow is depicted in Figure 1.

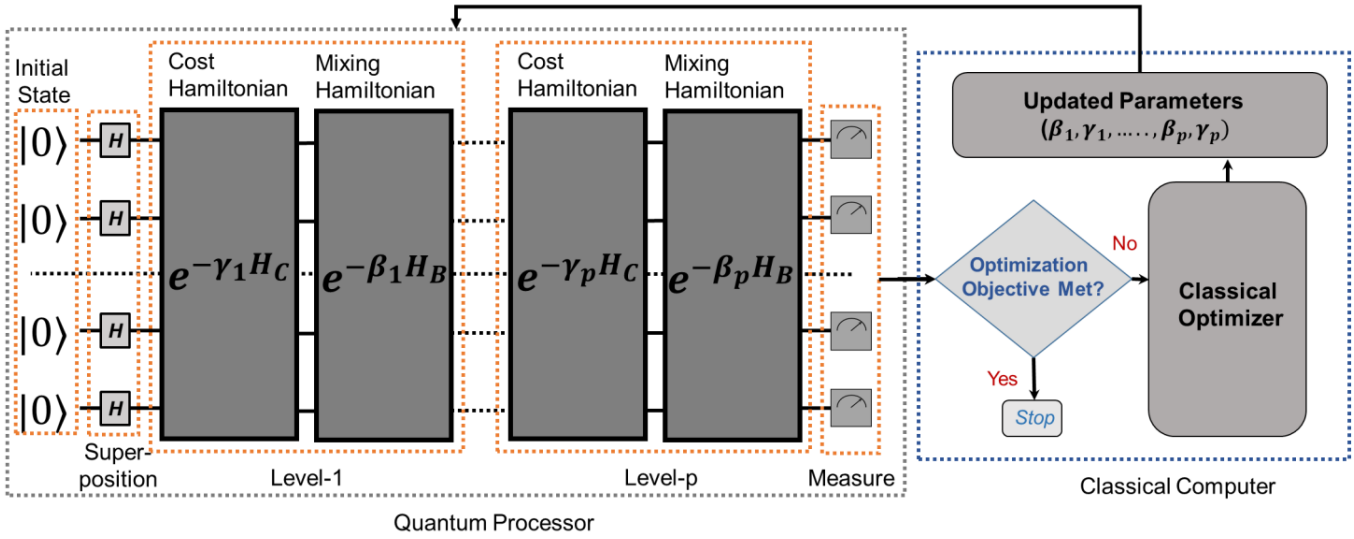


Figure 1: QAOA circuit diagram.

3 Solving an instance of the max-cut problem with QAOA

First of all, copy-past and run the following instructions in a new Python notebook:

```
!pip install qiskit
```

```
from qiskit.circuit.library import QAOAAnsatz
from qiskit.quantum_info import Statevector, SparsePauliOp
from qiskit.visualization import plot_histogram
from qiskit.primitives import StatevectorEstimator, StatevectorSampler
from scipy.optimize import minimize
import numpy as np
```

3.1 Cost Hamiltonian definition

In the case of the max-cut problem, the cost Hamiltonian is simply:

$$H_C = \sum_{(i,j)\in E} Z_i Z_j$$

and we are thus searching for the quantum state(s) on n qubits minimizing $\langle H_C \rangle$.

In the remaining of the document, we will consider the graph from Figure 2.

Activity 1

Instantiate a `SparsePauliOp` object recreating the problem cost Hamiltonian.

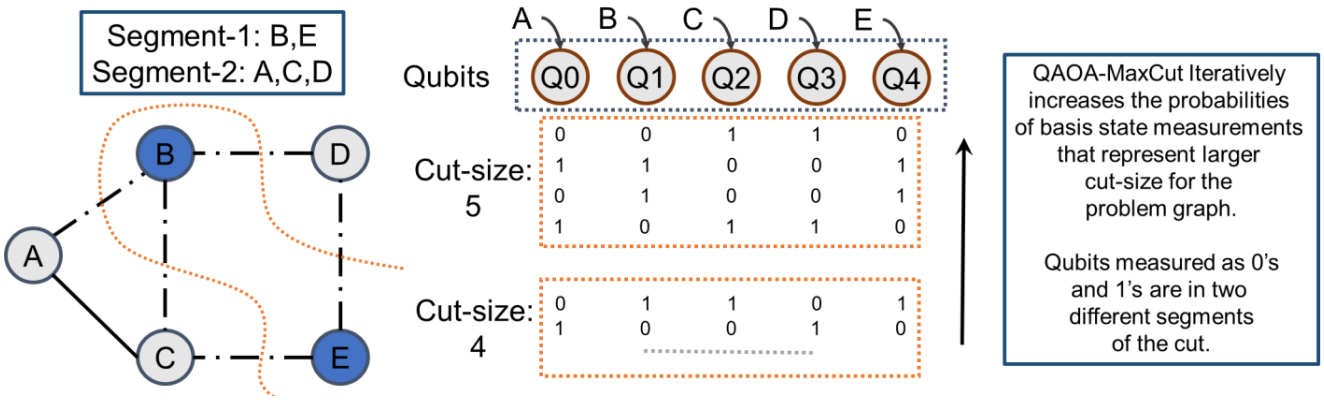


Figure 2: Using QAOA to solve the max-cut problem.

3.2 Exhaustive search

The studied graph being small, a first approach to solve the max-cut problem would consist in doing an exhaustive search on all computational basis states on five qubits so as to find the configuration(s) minimizing $\langle H_C \rangle$.

Activity 2

Implement an exhaustive search algorithm in Qiskit so as to find the configuration(s) minimizing $\langle H_C \rangle$ for the proposed graph. What is(are) this(these) configuration(s)? What is the size of the max-cut?

3.3 QAOA

Brute-force methods are non-efficient for larger graphs as the number of configurations explodes when increasing the number of vertices. An alternative approach consists then in solving the problem using QAOA.

3.3.1 Ansatz circuit

To implement QAOA on a digital quantum computer, having only access to single and two-qubit gates, U_{ansatz} has to be expressed as a gate sequence.

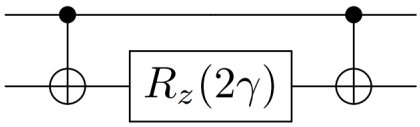
Regarding the evolution with the mixer Hamiltonian, the gate decomposition is quite obvious since the exponentiation a Pauli operator gives rise to a rotation gate. And so, the mixer Hamiltonian evolution $\mathrm{e}^{-i\beta[k]H_M}$ for the k-th layer corresponds, for each qubit recreating a vertex from the graph, to implementing a R_x gate of parameter $2\beta[k]$. The case of the cost Hamiltonian is slightly more complicated, let's have a look.

Activity 3

The inner loop of the algorithm first requires the application of the cost Hamiltonian evolution $\mathrm{e}^{-i\gamma[k]H_C}$ for the k-th layer. Show that this evolution corresponds, for each qubit pair reproducing an edge from the graph, to implementing the following operation:

$$U_C\big(\gamma[k]\big) = \begin{bmatrix} \mathrm{e}^{-i\gamma[k]} & 0 & 0 & 0 \\ 0 & \mathrm{e}^{i\gamma[k]} & 0 & 0 \\ 0 & 0 & \mathrm{e}^{i\gamma[k]} & 0 \\ 0 & 0 & 0 & \mathrm{e}^{-i\gamma[k]} \end{bmatrix}$$

Then verify thix matrix can be decomposed the following way:



Adding all these gates one by one to a `QuantumCircuit` object is time consuming. Hopefully, Qiskit comes with `QAOAAnsatz`, a generalized QAOA quantum circuit with a support of custom initial states and mixers. When instantiating an ansatz circuit using `QAOAAnsatz`, the following parameters must be specified:

- `cost_operator` – The operator representing the cost of the optimization problem;
- `reps` – The integer parameter p, which determines the number of layers in the circuit, default is 1;
- `initial_state` (optional) – An optional initial state to use. If `None` is passed then a set of Hadamard gates is applied as an initial state to all qubits;
- `mixer_operator` (optional) – An optional custom mixer to use instead of the global X-rotations;
- `name` (optional) – A name of the circuit, default 'qaoa'.

Activity 4

Instantiate an ansatz circuit of two layers - this hyper-parameter can later be changed - for the QAOA solver. The circuit can be displayed with different abstraction levels using first the `decompose` method, and then `draw`. For instance, `ansatz.decompose(reps=1).draw()` would display a block based representation of ansatz circuit `ansatz`. Increasing the value of `reps` in `decompose` will result in a more detailed representation - up to gate level.

3.3.2 Problem cost function

Having an ansatz circuit for QAOA, the next step consists in programming a problem cost function to minimize.

Activity 5

Build a (cost) function that takes as input the ansatz circuit, the cost Hamiltonian, as well as the circuit parameters (i.e. a `list` object of the form: $[\gamma_1, \beta_1, ..., \gamma_p, \beta_p]$), and returns $\langle\psi_p(\overrightarrow{\gamma}, \overrightarrow{\beta})|\, H_C \,|\psi_p(\overrightarrow{\gamma}, \overrightarrow{\beta})\rangle$.

3.3.3 Optimization

Any classical optimizer can be used to minimize the cost function. On a real quantum system, an optimizer designed for non-smooth cost function landscapes usually does better. In any case, we have to provide the optimizer with an initial guess (i.e. a starting point) for circuit parameters.

Activity 6

Build a NumPy array of length $2p$ of random parameters in between 0 and 2π.

We are finally ready to optimize the circuit parameters so as to find the graph portioning(s) maximizing the cut size. For that, one case use the `minimize` function from `SciPy`. It takes as input paramaters:

- `fun` – The cost function to be minimized;
- `x0` – Initial guess. Array of real elements of size $(n,)$, where n is the number of independent variables;
- `args` (optional) – A tuple containing extra arguments passed to the cost function;
- `method` – Type of solver.

In our case, `args` is a tuple containing the ansatz circuit and the cost Hamiltonian since those two objects are inputs of the problem cost function we have to specify in addition to the betas and the gammas.

Activity 7

Using the `"COBYLA"` optimizer, find a distribution of the betas and the gammas minimizing the problem cost function.

Activity 8

Instantiate a new QAOA circuit with solution parameters, add measurement operators, and then run multiple shots of this circuit so as to recreate a probability distribution. Are configurations corresponding to a max-cut dominating?